# INTEGRACAO: CAMPEONATOS E PLACAR ELETRONICO

2o Ano - Desenvolvimento de Sistemas | 3o trimestre | 19/10/2026
Guia para os grupos de Gerenciamento de Campeonatos e Placar Eletronico.
Repositorio-base: https://github.com/rvarjao/placar-eletronico-api

1. OBJETIVO E ARQUITETURA
Dois sistemas independentes trocam informacoes por API REST HTTP/JSON.
Campeonatos cria partidas, solicita um placar e consulta o resultado por ID.
Placar administra resultados, tempo, faltas, etapas e a interface visual.
Cada grupo usa seu codigo, projeto PostgreSQL e credenciais do Supabase.
Uma maquina da escola executa as APIs Flask em Docker na rede local.
O servidor precisa de internet para acessar o Supabase e portas liberadas.
Credenciais ficam no backend (.env), nunca no navegador ou no GitHub.

2. REPOSITORIO BASE - SEM LOGICA IMPLEMENTADA
https://github.com/rvarjao/placar-eletronico-api
O projeto ja traz app/routes/placares.py, services/, models/, repositories/,
Dockerfile, compose.yaml, .env.example, README e arquivos em docs/.
As rotas estao declaradas e retornam 501 (Not Implemented).
Os estudantes devem desenvolver validacoes, acesso ao banco e respostas.

3. ENDPOINTS
POST  /api/placares  cria placar (HTTP 201, retorna ID)
GET  /api/placares/{id}  consulta estado (HTTP 200 ou 404)
PATCH /api/placares/{id}  atualiza campos recebidos (HTTP 200)
Base ilustrativa: http://192.168.1.100:5001/api
Escrever os nomes de campos JSON em snake_case.

4. CAMPOS DO PLACAR
id: inteiro automatico gerado e persistido pela API.
nome_equipe_a, nome_equipe_b: textos obrigatorios.
modalidade: texto validado (futebol, futsal, basquete, volei, handebol, outro).
status: texto validado (nao_iniciado, em_andamento, pausado, finalizado).
tempo: inteiro em segundos; tipo de cronometro depende da modalidade.
etapa: texto livre (1o tempo, 2o tempo, 3o quarto, 4o set etc.).
pontos_equipe_a/b e faltas_equipe_a/b: inteiros nao negativos.
detalhes: objeto JSON para dados especificos (ex.: sets vencidos).

# ROTAS E CONTRATO JSON

5. CRIAR PLACAR - POST /api/placares

Entrada minima recomendada:

```
{ "nome_equipe_a": "Brasil", "nome_equipe_b": "Argentina",
  "modalidade": "futebol", "etapa": "1o tempo" }
```

Status inicial = nao_iniciado; tempo, pontos e faltas = 0.

Resposta esperada HTTP 201 Created:

```
{ "id": 15, "nome_equipe_a": "Brasil",
  "nome_equipe_b": "Argentina", "modalidade": "futebol",
  "status": "nao_iniciado", "tempo": 0, "etapa": "1o tempo",
  "pontos_equipe_a": 0, "pontos_equipe_b": 0,
  "faltas_equipe_a": 0, "faltas_equipe_b": 0, "detalhes": {} }
```

6. CONSULTAR PLACAR - GET /api/placares/15

Retorna o mesmo objeto, com valores atuais. Exemplo HTTP 200:

```
{ "id": 15, "nome_equipe_a": "Brasil",
  "nome_equipe_b": "Argentina", "modalidade": "futebol",
  "status": "em_andamento", "tempo": 2520, "etapa": "2o tempo",
  "pontos_equipe_a": 2, "pontos_equipe_b": 1,
  "faltas_equipe_a": 3, "faltas_equipe_b": 5, "detalhes": {} }
```

ID inexistente => HTTP 404 com mensagem JSON de erro.

7. ATUALIZAR PLACAR - PATCH /api/placares/15

Operacao do sistema do placar; exige autorizacao e validacao.

Enviar somente os campos alterados:

```
{ "pontos_equipe_a": 3, "tempo": 2700, "etapa": "2o tempo" }
```

Responder HTTP 200 com o objeto completo ja atualizado.

Nao permitir alterar o ID por PATCH. Validar pontuacoes e status.

8. TRATAMENTO DE ERROS

201: criado; 200: consulta/atualizacao; 400/422: JSON invalido.

401/403: sem autorizacao; 404: placar inexistente.

500: erro inesperado; 503: dependencia indisponivel.

Erros devem ter JSON coerente. Nunca expor senhas ou detalhes do banco.

# INTEGRACAO, INTERFACES E ENTREGAS

9. FLUXO DE INTEGRACAO - POLLING

1) Campeonatos cadastra a partida e executa POST /api/placares.

2) Salva o ID retornado como placar_id no proprio banco.

3) Consulta GET /api/placares/{id} aproximadamente a cada 5 segundos.

4) Mostra pontos, tempo, etapa, faltas e detalhes na sua interface.

5) Ao receber status finalizado, para o polling e registra resultado final.

Em falha/timeout, mostrar aviso e tentar novamente, sem criar outro placar.

Para fins didaticos, usar polling; webhook e WebSocket nao sao exigidos.

10. INTERFACES VISUAIS POR MODALIDADE

Primeiro criar um placar generico; depois reutilizar componentes e

adaptar layouts por modalidade sem duplicar toda a aplicacao.

Futebol/futsal: gols, 1o/2o tempo, cronometro crescente; faltas se couber.

Basquete: pontos, quartos, cronometro regressivo e faltas.

Volei: pontos do set atual, sets vencidos e etapa; relogio opcional.

No volei, pontos_equipe_a/b representam pontos do set em andamento.

Exemplo detalhes: { "sets_equipe_a": 2, "sets_equipe_b": 1 }

O sistema de campeonatos deve interpretar esses dados corretamente.

11. ETAPAS PROPOSTAS

1. Fazer fork do projeto base e estudar o contrato JSON.

2. Modelar tabelas, configurar Supabase e testar conexao pelo backend.

3. Implementar POST, GET e PATCH, validacoes e tratamento de erros.

4. Criar controles e tela de visualizacao do placar generico.

5. Integrar os dois projetos pela URL da rede local.

6. Especializar a interface para futebol, basquete e volei.

7. Criar testes automatizados e demonstrar partida ate finalizacao.

12. CRITERIOS DE ACEITACAO

POST deve retornar ID persistente; GET refletir mudancas ou retornar 404.

PATCH deve validar dados e impedir alteracoes nao autorizadas.

Campeonatos armazena ID e acessa somente a API, nao o banco alheio.

As interfaces devem respeitar a modalidade e exibir resultado correto.

Testar indisponibilidade de rede/Supabase e documentar o comportamento.

DOCUMENTACAO COMPLEMENTAR

https://github.com/rvarjao/placar-eletronico-api/tree/main/docs